



Image, sécurité et deep learning

Compte-rendu TP6

Insertion de données cachées 3D

Louis Jean

Master 2 IMAGINE
Université de Montpellier
N° étudiant : 21914083

14 octobre 2024

Table des matières

1	Introduction	2
2	Méthode de Cho	2
2.1	Insertion du message	2
2.2	Extraction du message	3
3	Implémentation de la méthode de Cho	3
4	Résultats obtenus	3
5	Variation des paramètres	5
5.1	RMSE en fonction de α	5
5.2	RMSE en fonction de n	6
6	Proposition d'attaque sur la méthode de Cho	6
7	Conclusion	6

1 Introduction

Dans ce travail, nous explorons la méthode de Cho, une technique de tatouage d'objets 3D. Elle permet d'insérer un message caché dans un maillage 3D de façon imperceptible en modifiant les coordonnées des sommets, tout en cherchant à minimiser les modifications visibles.

2 Méthode de Cho

La méthode de Cho est une technique de tatouage d'objets 3D qui repose sur la modification imperceptible des coordonnées des sommets d'un maillage.

2.1 Insertion du message

Le principe est d'insérer le message en modifiant les coordonnées des sommets sélectionnés du maillage :

- On choisit α qui définit l'intensité des modifications appliquées aux normes des sommets
- On calcule le centre de gravité g du maillage en faisant la moyenne des coordonnées de tous les sommets
- On convertit les coordonnées des sommets en coordonnées sphériques en prenant g comme origine
- Depuis ces coordonnées sphériques, on peut récupérer les normes des vecteurs de chaque sommet
- On trie les sommets en fonction de leurs normes et on les divise en plusieurs groupes de tailles égales, le nombre de groupes correspond à la longueur du message à insérer
- On normalise les normes dans chaque groupe pour les ramener dans l'intervalle $[0, 1]$
- On encode le message (watermark) que l'on veut cacher en bits, on dira qu'un bit à 0 correspond -1 et un bit à 1 correspond +1
- On calcule ensuite la moyenne de chaque groupe
- Si le bit du watermark associé au groupe vaut +1 :
 - Si la moyenne est supérieure à $\frac{1}{2} + \alpha$, on passe au groupe suivant
 - Sinon, on décrémente k , on élève chaque valeur du groupe à la puissance k et on recommence jusqu'à ce que la moyenne soit supérieure à $\frac{1}{2} + \alpha$
- Si le bit du watermark associé au groupe vaut -1 :
 - Si la moyenne est inférieure à $\frac{1}{2} - \alpha$, on passe au groupe suivant
 - Sinon, on incrémente k , on élève chaque valeur du groupe à la puissance k et on recommence jusqu'à ce que la moyenne soit inférieure à $\frac{1}{2} - \alpha$
- On remappe les normes modifiées dans leurs plages d'origine respectives pour chaque groupe
- On reconvertit les coordonnées sphériques modifiées en coordonnées cartésiennes
- On ajoute le centre de gravité g aux coordonnées des sommets pour enfin obtenir le maillage tatoué

2.2 Extraction du message

Pour extraire le message du maillage tatoué, on suit un processus inverse à l'insertion.

- On calcule (ou on utilise) le même centre de gravité que lors de l'insertion
- On convertit les coordonnées cartésiennes des sommets du maillage tatoué en coordonnées sphériques par rapport à g
- Depuis ces coordonnées sphériques, on peut récupérer les normes des vecteurs de chaque sommet
- On trie les sommets en fonction de leurs normes et on les divise en groupes de tailles égales, en utilisant le même nombre de groupes que lors de l'insertion
- On normalise les normes dans chaque groupe pour les ramener dans l'intervalle $[0, 1]$
- On calcule la moyenne de chaque groupe
- Si la moyenne est supérieure à $\frac{1}{2}$, on rajoute un $+1$ dans le message en cours de décodage
- Si la moyenne est inférieure à $\frac{1}{2}$, on rajoute un -1 dans le message en cours de décodage
- À la fin du processus, on retrouve le message caché dans le maillage

3 Implémentation de la méthode de Cho

J'ai choisi d'implémenter cette méthode en `Python` car c'est plus simple et moins long, surtout dans le cas d'un petit exemple pour un TP. Dans le cadre d'une vraie application utilisant cet algorithme, j'aurais évidemment choisi `C++`. Pour observer les rendus des maillages, j'ai simplement utilisé l'extension `3D Viewer` dans `VSCodium`.

4 Résultats obtenus

Pour tester cette méthode, j'ai choisi un modèle *bonfire.obj*, qui représente un objet phare de la licence de jeux vidéo *Dark Souls*.



Figure 1: *bonfire.obj*

J'ai choisi de prendre une séquence aléatoire de valeurs pour mon watermark (soit -1 soit +1), de taille n . Voici quelques résultats pour différents n et différents α .

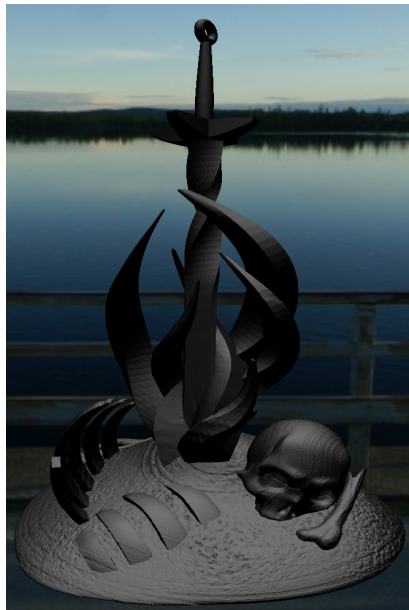
n	alpha	Modification maximale d'un sommet	Modification moyenne des sommets
100	0.5	41.3	0.79
100	0.1	13.2	0.28
100	0.05	0.09	0.02
10	0.05	2.73	0.57
10	0.1	43.37	4.18
10	0.5	87.19	3.46

Table 1: Modifications des sommets en fonction de n et α

Globalement, dans mon cas, on remarque qu'une petite taille de message ou qu'un α trop grand coûte cher sur la qualité du maillage tatoué. Cela s'explique certainement par le fait que le nombre de groupes est déterminé par n , et que l'intensité des modifications des sommets est induite par α .



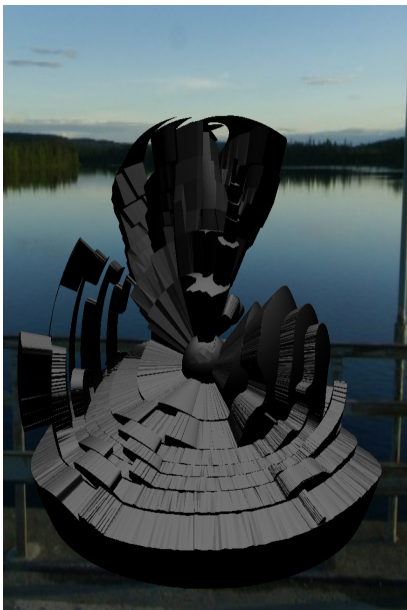
(a) $n = 100, \alpha = 0,5$



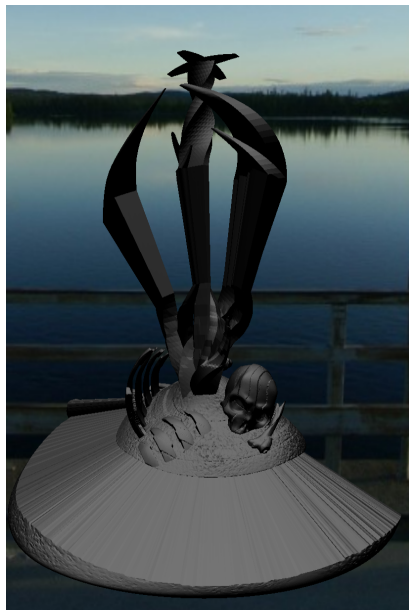
(b) $n = 100, \alpha = 0,1$



(c) $n = 100, \alpha = 0,05$



(d) $n = 10, \alpha = 0,5$



(e) $n = 10, \alpha = 0,1$



(f) $n = 10, \alpha = 0,05$

Figure 2: Maillages tatoués pour différentes valeurs de n et de α

Je précise que dans tous les cas ci-dessus, je parvenais bien à retrouver le message caché.

5 Variation des paramètres

Il était proposé d'étudier certaines métriques en fonction de paramètres de l'algorithme, comme la distance RMSE, qui s'écrit ainsi :

$$RMSE = \sqrt{\frac{1}{k} \sum_{i=0}^k (v'_i - v_i)^2}$$

5.1 RMSE en fonction de α

J'ai tracé la courbe de la distance RMSE en fonction du paramètre α . On observe que la fonction est linéaire.

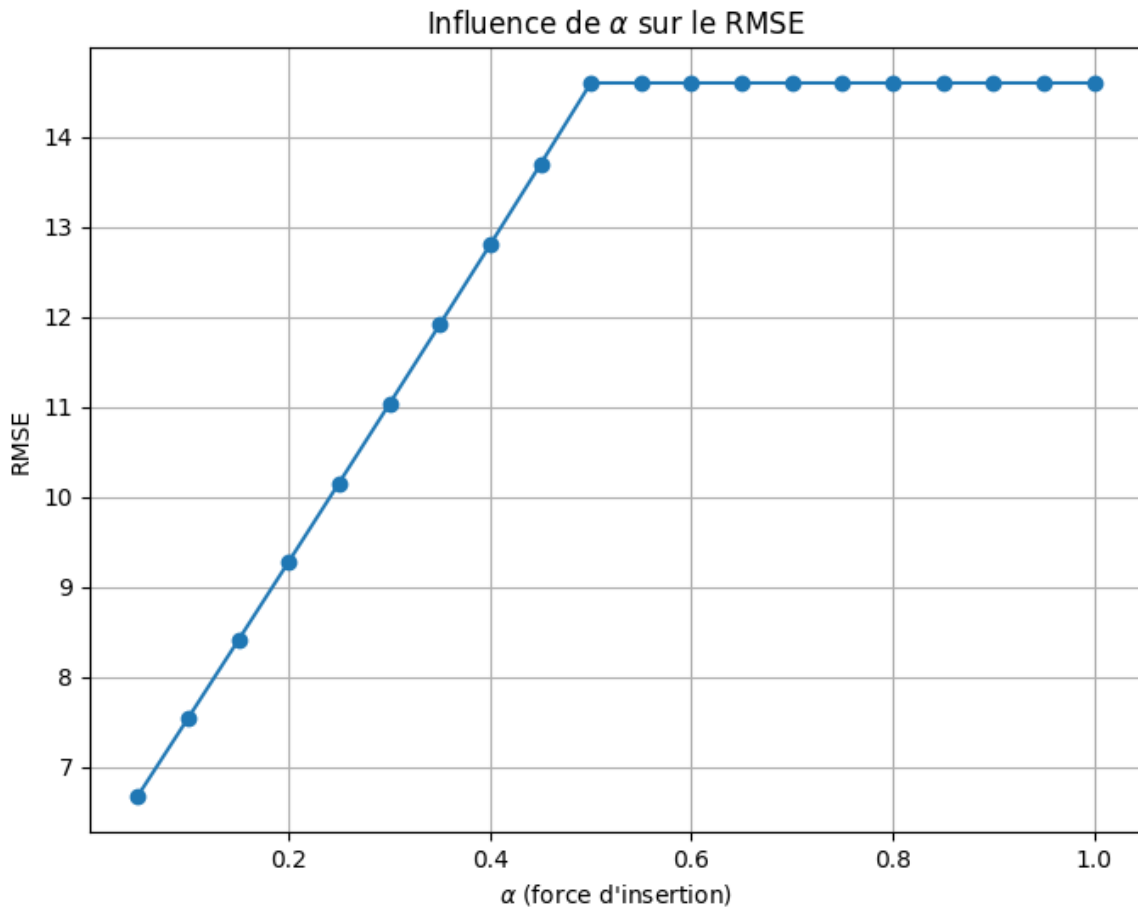
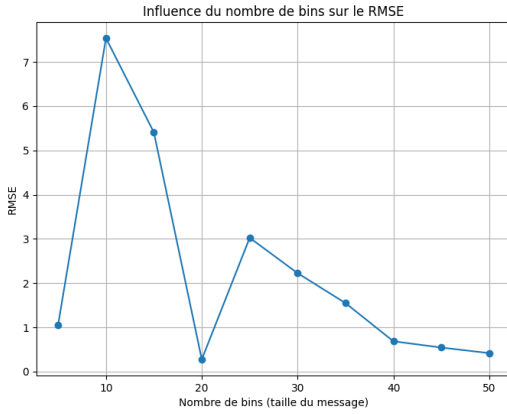


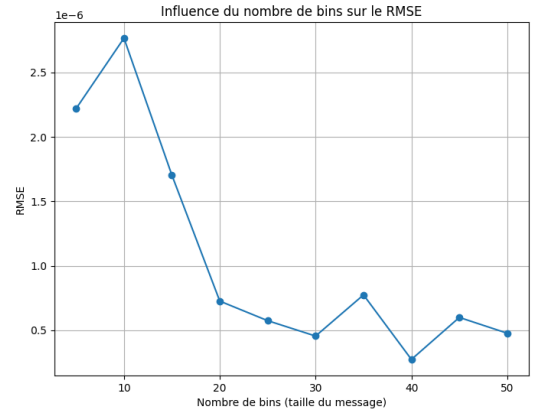
Figure 3: RMSE en fonction de α

5.2 RMSE en fonction de n

En traçant la distance RMSE en fonction de n (le nombre de bins = nombre de groupes = taille du message), on se rend compte que la tendance de la fonction est de décroître lorsque le nombre de bins augmente. De là à dire qu'elle est presque inversement proportionnelle au nombre de bins, il n'y a qu'un pas. Je pense que ceci s'explique par le fait suivant : plus il y a de bins, moins les dits bins contiennent de sommets, et donc chaque modification est appliquée à un plus petit nombre de sommets, ce qui fait que les ajustements sont répartis sur un plus grand nombre de sommets, et donc qu'au final à l'œil nu on n'y voit que du feu.



(a) *bonfire.obj*



(b) *sphere.obj*

Figure 4: RMSE en fonction de n , pour deux maillages différents

6 Proposition d'attaque sur la méthode de Cho

Pour attaquer la robustesse de la méthode de Cho, on pourrait lisser le maillage (par exemple avec un lissage Laplacien comme nous venons de faire dans le dernier TP de M^{me} Faraj), et ainsi agir directement sur les normes des vecteurs, rendant le message plus difficile si ce n'est impossible à récupérer à l'arrivée.

7 Conclusion

En conclusion, ce travail a montré que la méthode de Cho est efficace pour insérer un message de manière discrète dans un maillage 3D, mais n'est pas exempte de défauts ni de vulnérabilités. J'ai bien apprécié la liberté laissée dans le TP.

Merci pour le temps et l'attention que vous avez consacrés à la lecture de ce compte-rendu, **mes plus plates excuses pour le retard dans le rendu de ce travail.**